Java Template

```
import java.util.*;
import java.io.*;

public class JavaTemplate {

  public static void main(String[] args) throws Exception {
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    // BufferedReader br = new BufferedReader(new FileReader("testfile.in"));
    PrintWriter pw = new PrintWriter(System.out);

    StringTokenizer st = new StringTokenizer(br.readLine());
    int T = Integer.parseInt(st.nextToken());

    int[] arr = new int[T];
    ArrayList<Integer> arrLs = new ArrayList<Integer>();

    pw.close();
  }

}
```

Python Template

```
a, b, c = map(int, input().split())
print("The sum of these three numbers is", a + b + c)
```

Floodfill

```
public class Floodfill {
	private static int[][] grid;  // the grid itself
	private static int rowNum;
	private static int colNum;	// grid dimensions, rows and columns
	private static boolean[][] visited;  // keeps track of which nodes have been
					// visited
	private static int currSize = 0;  // reset to 0 each time we start a new component

	public static void main(String[] args) {
		/*
		 * input code and other problem-specific stuff here
		 */
		for (int r = 0; r < rowNum; r++) {
			for (int c = 0; c < colNum; c++) {
				if (!visited[r][c]) {
					currSize = 0;
					/*
					 * start a flood fill if the square hasn't already been
					 * visited, and then store or otherwise use the component
					 * size for whatever it's needed for
					 */
					floodfill(r, c, grid[r][c]);
				}
			}
		}
	}

	private static void floodfill(int r, int c, int color) {
		if ((r < 0 || r >= rowNum || c < 0 || c >= colNum)  // if out of bounds
		   || grid[r][c] != color			// wrong color
		   || visited[r][c]  // already visited this square
		)
```

```
                return;

            visited[r][c] = true;  // mark current square as visited
            currSize++;          // increment the size for each square we visit

            // recursively call flood fill for neighboring squares
            floodfill(r, c + 1, color);
            floodfill(r, c - 1, color);
            floodfill(r - 1, c, color);
            floodfill(r + 1, c, color);
        }
}
```

Binary Search on Target

```
def binary_search(arr: list[int], target: int) -> int:
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Advanced Binary Search

```
def binary_search_boundary(arr: list) -> int:
    def condition(value) -> bool:
        pass  # Returns True/False evaluation

    left, right = 0, len(arr)
    while left < right:
        mid = left + (right - left) // 2
        if condition(mid):
            right = mid
        else:
            left = mid + 1
    return left
```

Arrays Functions *(.sort() and .binarySearch())*

```
import java.util.Arrays;

public class ArrayExample {
    public static void main(String[] args) {
        String[] employees = {"Charlie", "Bob", "Alice", "Dan"};
```

```
    // 1. Sort it first!
    Arrays.sort(employees); // Now: ["Alice", "Bob", "Charlie", "Dan"]

    // 2. Search it!
    int targetIndex = Arrays.binarySearch(employees, "Bob");

    if (targetIndex >= 0) {
      System.out.println("Found Bob at position " + targetIndex);
    } else {
      System.out.println("Employee not found.");
    }
  }
}
```

2D Matrix

```
int[][] emptyMatrix = new int[2][3];

// Rows are accessed via .length, columns via matrix[i].length
for (int i = 0; i < matrix.length; i++) {
  for (int j = 0; j < matrix[i].length; j++) {
    System.out.print(matrix[i][j] + " ");
  }
  System.out.println();
}
```

Kadane's Algorithm

```
def maxSubarraySum(arr):

    # Stores the result (maximum sum found so far)
    res = arr[0]

    # Maximum sum of subarray ending at current position
    maxEnding = arr[0]

    for i in range(1, len(arr)):

        # Either extend the previous subarray or start
        # new from current element
        maxEnding = max(maxEnding + arr[i], arr[i])

        # Update result if the new subarray sum is larger
        res = max(res, maxEnding)

    return res

if __name__ == "__main__":
    arr = [2, 3, -8, 7, -1, 2, 3]
    print(maxSubarraySum(arr))
```

DFS

Depth-first search (DFS) is a straightforward graph traversal technique. The algorithm begins at a starting node, and proceeds to all other nodes that are reachable from the starting node using the edges of the graph.

Depth-first search always follows a single path in the graph as long as it finds new nodes. After this, it returns to previous nodes and begins to explore other parts of the graph. The algorithm keeps track of visited nodes, so that it processes each node only once.

```
import sys

sys.setrecursionlimit(10**5)  # Python has a default recursion limit of 1000

n = 6
visited = [False] * n

"""
Define adjacency list and read in problem-specific input here.

In this example, we've provided "dummy input" that's
reflected in the GIF above to help illustrate the
order of the recursive calls.
"""
adj = [[] for _ in range(n)]
adj[0] = [1, 2, 4]
adj[1] = [3, 4]
adj[2] = [5]


def dfs(current_node):
    visited[current_node] = True
    for neighbor in adj[current_node]:
        if not visited[neighbor]:
            dfs(neighbor)


for i in range(n):
    if not visited[i]:
        dfs(i)
```

```
import java.io.*;
import java.util.*;

public class DFSDemo {
	static List<Integer>[] adj;
	static boolean[] visited;
	static int n = 6;

	public static void main(String[] args) throws IOException {
		visited = new boolean[n];

		/*
		 * Define adjacency list and read in problem-specific input
		 *
		 * In this example, we've provided "dummy input" that's
		 * reflected in the GIF above to help illustrate the
		 * order of the recursive calls.
		 */
		adj = new ArrayList[n];
		for (int i = 0; i < n; i++) { adj[i] = new ArrayList<>(); }
		adj[0] = new ArrayList<>(Arrays.asList(1, 2, 4));
		adj[1] = new ArrayList<>(Arrays.asList(3, 4));
		adj[2] = new ArrayList<>(Arrays.asList(5));

		for (int i = 0; i < n; i++) {
			// iterate over all connected components in the graph
			if (!visited[i]) { dfs(i); }
		}
	}

	static void dfs(int currentNode) {
		if (visited[currentNode]) { return; }
```

```
            visited[currentNode] = true;
            for (int neighbor : adj[currentNode]) { dfs(neighbor); }
        }
}
```

BFS

In a breadth-first search, we travel through the vertices in order of their distance from the starting vertex.

```
from collections import deque

"""
Define adjacency list and read in problem-specific input

In this example, we've provided "dummy input" that's
reflected in the GIF above to help illustrate the
order of the recursive calls.
"""

n = 6
adj = [[] for _ in range(n)]
visited = [False] * n

adj[0] = [1, 2, 4]
adj[1] = [3, 4]
adj[2] = [5]

for i in range(n):
    # iterate over all connected components in the graph
    if not visited[i]:
        q = deque([i])

        while q:
            node = q.popleft()
            visited[node] = True

            for neighbor in adj[node]:
                if not visited[neighbor]:
                    q.append(neighbor)
```

```
import java.util.*;

public class BFSDemo{
	public static void main(String[] args) {
		int n = 6;
		boolean[] visited = new boolean[n];
		List<Integer>[] adj = new ArrayList[6];
		for (int i = 0; i < n; i++) { adj[i] = new ArrayList<>(); }
		adj[0] = new ArrayList<>(Arrays.asList(1, 2, 4));
		adj[1] = new ArrayList<>(Arrays.asList(3, 4));
		adj[2] = new ArrayList<>(Arrays.asList(5));

		/*
		 * Define adjacency list and read in problem-specific input
		 *
		 * In this example, we've provided "dummy input" that's
		 * reflected in the GIF above to help illustrate the
		 * order of the recursive calls.
		 */
		for (int i = 0; i < n; i++) {
			// iterate over all connected components in the graph
			if (!visited[i]) {
				Queue<Integer> q = new LinkedList<Integer>();
				q.add(i);

				while (q.size() > 0) {
					int currentNode = q.peek();
					q.poll();

					visited[currentNode] = true;

					for (int neighbor : adj[currentNode]) {
						if (!visited[neighbor]) { q.add(neighbor); }
```

```
                                    }
                                }
                            }
                        }
                    }
}
```

ArrayDeque

Copy JAVA

```java
ArrayDeque<Integer> deque = new ArrayDeque<Integer>();
deque.addFirst(3);    // [3]
deque.addFirst(4);    // [4, 3]
deque.addLast(7);     // [4, 3, 7]
deque.removeFirst();  // [3, 7]
deque.addFirst(1);    // [1, 3, 7]
deque.removeLast();   // [1, 3]
```

HashSet in Python

Method	Explanation	Example
set()	*Creates a new set*	my_set = {1, 2, 3}
.add(element)	*Adds a single element to the set. If it already exists, nothing changes.*	my_set.add(4)
.update(iterable)	*Adds multiple elements from any iterable (list, tuple, another set).*	my_set.update([5, 6])
.remove(element)	*Removes the element. Raises a KeyError if the element is not found.*	my_set.remove(2)
.discard(element)	*Removes the element if it exists. Does not raise an error if missing.*	my_set.discard(99)
.pop()	*Removes and returns an arbitrary element from the set.*	item = my_set.pop()
.clear()	*Removes all elements, leaving the set completely empty.*	my_set.clear()

HashSet in Java

```
import java.util.HashSet;

public class HashSetExample {
   public static void main(String[] args) {
      // Create a HashSet
      HashSet<String> programmingLanguages = new HashSet<>();

      // 1. add()
      programmingLanguages.add("Java");
      programmingLanguages.add("Python");
      programmingLanguages.add("C++");
      boolean isAdded = programmingLanguages.add("Java"); // Will return false (duplicate)

      // 2. contains()
      System.out.println("Contains Python? " + programmingLanguages.contains("Python")); // true

      // 3. size()
      System.out.println("Set size: " + programmingLanguages.size()); // 3

      // 4. remove()
      programmingLanguages.remove("C++");

      // 5. isEmpty()
      System.out.println("Is set empty? " + programmingLanguages.isEmpty()); // false

      // 6. Iterating using an enhanced for-loop
      System.out.println("Remaining items:");
      for (String lang : programmingLanguages) {
         System.out.println(lang);
      }

      // 7. clear()
      programmingLanguages.clear();
      System.out.println("Size after clear: " + programmingLanguages.size()); // 0
```

```
    }
  }
  }
```

HashMap in Python

```
# 1. Initialize the dictionary with numeric keys and values
# Key: Student ID (int) -> Value: Exam Score (int)
scores = {101: 85, 102: 92}
print("Initial scores:", scores)


# 2. Use .get() to read a score safely
# Student 103 hasn't taken the test yet, so we return a default score of 0.
student_103_score = scores.get(103, 0)
print(f"\n[get] Student 103 score: {student_103_score}")


# 3. Use [] assignment to add or change a score
scores[103] = 78  # Adds a new student ID and score
scores[101] = 88  # Updates Student 101's score after a regrade
print("\n[[] assignment] Added student 103 and updated 101:")
print(scores)


# 4. Use .update() to process multiple scores at once
# We process a batch of new test submissions.
batch_scores = {104: 95, 105: 81, 102: 96}
scores.update(batch_scores)
print("\n[.update()] Processed a batch of new scores:")
print(scores)

scores = {101: 88, 102: 96, 103: 78}

for student_id, score in scores.items():
   print(f"Student ID {student_id} earned a score of {score}")

for student_id in scores.keys():
   print(f"Present Student ID: {student_id}")
```

```
for score in scores.values():
  print(f"Value: {score}")
```

- **`put(K key, V value)`**: Inserts a key-value pair into the map. If the key already exists, its old value is overwritten and returned.
- **`get(Object key)`**: Returns the value associated with the specified key, or `null` if the key does not exist.
- **`getOrDefault(Object key, V defaultValue)`**: Returns the value of the key, or a fallback default value if the key is missing.
- **`remove(Object key)`**: Removes the key-value mapping for the specified key.
- **`clear()`**: Removes all mappings, completely emptying the `HashMap`.
- **`size()`**: Returns the total number of key-value pairs currently in the map.
- **`isEmpty()`**: Returns `true` if the map contains no elements.
- **`containsKey(Object key)`**: Returns `true` if the map contains the specified key.
- **`containsValue(Object value)`**: Returns `true` if one or more keys map to the specified value.

```
for (String key : map.keySet()) {

   System.out.println("Key: " + key);

}

for (Integer value : map.values()) {

   System.out.println("Value: " + value);

}
```

HashMap in Java

```
import java.util.HashMap;

import java.util.Map;



public class HashMapExample {

    public static void main(String[] args) {

        // Initialization (Requires wrapper classes like Integer instead of int)

        HashMap<String, Integer> inventory = new HashMap<>();



        // 1. Adding elements using put()

        inventory.put("Apple", 50);

        inventory.put("Banana", 20);

        inventory.put("Orange", 35);



        // 2. Retrieving elements using get() and getOrDefault()

        int appleCount = inventory.get("Apple"); // Returns 50

        int grapeCount = inventory.getOrDefault("Grape", 0); // Returns 0 (not found)
```

```
// 3. Checking presence
boolean hasBanana = inventory.containsKey("Banana"); // true

// 4. Updating a value
inventory.put("Apple", 60); // Overwrites 50 with 60

// 5. Removing an item
inventory.remove("Orange");

// 6. Iterating using entrySet()
for (Map.Entry<String, Integer> entry : inventory.entrySet()) {
    System.out.println(entry.getKey() + " Quantity: " + entry.getValue());
}

// Check size
System.out.println("Total unique items: " + inventory.size());
```

```
    }

}
```

Queue

```
import java.util.Queue;

import java.util.ArrayDeque;



public class QueueExample {

    public static void main(String[] args) {

        // 1. Instantiate using ArrayDeque

        Queue<String> line = new ArrayDeque<>();



        // 2. Enqueue (Add elements to the tail)

        line.offer("Alice");

        line.offer("Bob");

        line.offer("Charlie");

        System.out.println("Queue: " + line); // Output: [Alice, Bob, Charlie]



        // 3. Inspect (Look at the head without removing)
```

```
        String nextUp = line.peek();

        System.out.println("Next in line: " + nextUp); // Output: Alice



        // 4. Dequeue (Remove and return the head)

        String served = line.poll();

        System.out.println("Served: " + served); // Output: Alice



        System.out.println("Remaining Queue: " + line); // Output: [Bob, Charlie]

    }

}
```

Stack

```
import java.util.Deque;

import java.util.ArrayDeque;



public class StackExample {

    public static void main(String[] args) {

        // Instantiate using Deque and ArrayDeque

        Deque<String> bookStack = new ArrayDeque<>();
```

```
        // 1. PUSH (Add element to the top)

        bookStack.push("Harry Potter");

        bookStack.push("The Hobbit");

        bookStack.push("1984");

        System.out.println("Stack: " + bookStack); // Output: [1984, The Hobbit, Harry
Potter]



        // 2. PEEK (Look at the top element without removing)

        System.out.println("Top book: " + bookStack.peek()); // Output: 1984



        // 3. POP (Remove and return the top element)

        String removedBook = bookStack.pop();

        System.out.println("Popped: " + removedBook); // Output: 1984



        System.out.println("Remaining Stack: " + bookStack); // Output: [The Hobbit, Harry
Potter]

    }
```

```
}
```

ArrayList

```
import java.util.ArrayList;

public class Main {

    public static void main(String[] args) {

        // Create an ArrayList

        ArrayList<String> fruits = new ArrayList<>();



        // 1. Add elements

        fruits.add("Apple");

        fruits.add("Banana");

        fruits.add(1, "Mango"); // Inserts at index 1



        // 2. Access and modify

        String firstFruit = fruits.get(0); // "Apple"

        fruits.set(2, "Blackberry");      // Replaces "Banana" with "Blackberry"
```

```
        // 3. Information
        int totalItems = fruits.size();    // 3
        boolean hasApple = fruits.contains("Apple"); // true


        // 4. Remove elements
        fruits.remove(1);               // Removes "Mango"
        fruits.remove("Apple");         // Removes "Apple"


        // 5. Functional Loop (Java 8+)
       for (String fruit : fruits) {
           System.out.println("Fruit name: " + fruit);
        }
    }
}
```

Arrays

```
import java.util.Arrays;
```

Java

```
public class JavaArrays {

    public static void main(String[] args) {

        // Option A: Fixed size, empty initialization (defaults to 0)

        int[] scores = new int[5];

        scores[0] = 90;

        scores[1] = 85;



        // Option B: Inline literal initialization

        String[] fruits = {"Apple", "Banana", "Mango"};



        // 1. Get Length (It is a property, NOT a method: no parentheses!)

        int size = fruits.length; // 3



        // 2. Read / Write Elements

        String first = fruits[0]; // "Apple"

        fruits[2] = "Blueberry";  // Replaces "Mango"
```

```
        // 3. Printing an Array
        System.out.println(Arrays.toString(fruits)); // ["Apple", "Banana", "Blueberry"]
    }
}
```

Python

```
# Option A: Universal Python List (Acts as a dynamic array)
fruits = ["Apple", "Banana", "Mango"]

# Option B: True Strict Primitive Array (Built-in module)
import array
# 'i' means signed integers only. Throws TypeError if you try to add a string.
scores = array.array('i', [90, 85, 77])

# 1. Get Length
size = len(fruits) # 3

# 2. Read / Write Elements
first = fruits[0]       # "Apple"
```

```
fruits[2] = "Blueberry"  # Replaces "Mango"



# 3. Dynamic Operations (Only works on vanilla lists, not primitive Java arrays)

fruits.append("Orange")  # Adds to the end

fruits.pop(1)          # Removes index 1 ("Banana")
```

1. Fast Odd/Even Check (`& 1`)

Instead of using the modulo operator (`% 2`), you can check the very last bit of a number. If the last bit is `1`, the number is odd. If it is `0`, the number is even.

- **Why use it?** It is faster than division/modulo at the hardware level.

java

None

```
int num = 45;



if ((num & 1) == 0) {

    System.out.println("Even");

} else {

    System.out.println("Odd"); // 45 is binary 101101 (ends in 1, so
Odd)
```

```
}
```

2. Status Flags & Permissions (Bit Masking `|`, `&`)

Instead of creating 4 or 5 separate `boolean` variables, you can store multiple configurations inside a single `int`. Each bit represents a different permission (Read, Write, Execute).

java

None

```
// Define the bit positions (powers of 2)

final int READ    = 1; // 0001

final int WRITE   = 2; // 0010

final int EXECUTE = 4; // 0100



// 1. Combine permissions using OR (|)

int userPermissions = READ | WRITE; // 0011 (Can read and write)



// 2. Check if a user has a specific permission using AND (&)

boolean canWrite   = (userPermissions & WRITE) != 0;   // true (0011 &
0010 != 0)

boolean canExecute = (userPermissions & EXECUTE) != 0; // false (0011
& 0100 == 0)
```

3. Swapping Two Numbers Without a Temporary Variable (^)

You can swap two integers using the XOR operator without allocating a third variable to temporarily hold a value.

- **How it works:** XORing a value with the same value twice cancels it out.

java

```
None
int x = 10;

int y = 5;



x = x ^ y;

y = x ^ y; // y becomes 10

x = x ^ y; // x becomes 5



System.out.println("x: " + x + ", y: " + y); // x: 5, y: 10
```

.4. Fast Multiplication and Division (<<, >>)

Shifting bits left multiplies a number by 2. Shifting bits right divides a number by 2.

- **Why use it?** This is a classic optimization trick in game development (like calculating grid movements or pixels) because bit-shifting is incredibly light on the CPU.

java

None

```
int speed = 20;



int doubleSpeed = speed << 1; // 20 * 2 = 40

int halfSpeed   = speed >> 1; // 20 / 2 = 10

int quadSpeed   = speed << 2; // 20 * 4 = 80
```

5. Finding the Single Unique Number in an Array (`^`)

A classic LeetCode problem: You are given an array where every element appears twice except for one. Find that single one.

- **How it works:** Because `X ^ X = 0` and `X ^ 0 = X`, if you XOR every number together, all the duplicates perfectly erase each other, leaving only the unique number.

java

None

```
int[] nums = {4, 1, 2, 1, 2};

int unique = 0;



for (int num : nums) {

    unique ^= num;

}
```

```
System.out.println("Unique number: " + unique); // Outputs 4
```

Base Conversions

Python:

==

1. BASE 10 ---> BASE 2 (Decimal to Binary)

==

decimal_num = 13

Built-in method: bin()

Note: This returns a string starting with '0b' (e.g., '0b1101')

binary_str_with_prefix = bin(decimal_num)

Clean string without the '0b' prefix

binary_str = format(decimal_num, 'b')

print(f"Decimal 13 to Binary: {binary_str}") # Output: "1101"

```
# ========================================

# 2. BASE 2 ---> BASE 10 (Binary to Decimal)

# ========================================

binary_input = "1101"



# Built-in method: int(string, base)

decimal_output = int(binary_input, 2)

print(f"Binary 1101 to Decimal: {decimal_output}") # Output: 13
```

Java:

```
public class BaseConversion {

    public static void main(String[] args) {

        // ========================================

        // 1. BASE 10 ---> BASE 2 (Decimal to Binary)

        // ========================================

        int decimalNum = 13;
```

```
        // Built-in method: Integer.toBinaryString(int)
        String binaryStr = Integer.toBinaryString(decimalNum);
        System.out.println("Decimal 13 to Binary: " + binaryStr); // Output: "1101"



        // ========================================
        // 2. BASE 2 ---> BASE 10 (Binary to Decimal)
        // ========================================
        String binaryInput = "1101";

        // Built-in method: Integer.parseInt(string, base)
        int decimalOutput = Integer.parseInt(binaryInput, 2);
        System.out.println("Binary 1101 to Decimal: " + decimalOutput); // Output: 13
    }
}
```

- **To Decimal (Base 10):**
  - **Python:** `int("string", base)`
  - **Java:** `Integer.parseInt("string", base)`
- **From Decimal (Base 10):**
  - **Python:** Requires a custom modulo-division loop (shown above).
  - **Java:** `Integer.toString(number, base)`

Any Base → Base 10 (Decimal)

The built-in `int(string, base)` function dynamically accepts any base from **2 to 36**.

python

```python
# Base 5 string ('42' in base 5 = 4*5^1 + 2*5^0 = 22)
print(int("42", 5))   # Output: 22

# Base 36 string ('zz' in base 36 = 35*36^1 + 35*36^0 = 1295)
print(int("zz", 36))  # Output: 1295
```

Use code with caution.

Base 10 (Decimal) → Any Base

Python does not have a built-in method to format a decimal into a custom base string natively. You must write a lightweight math loop to handle it:

python

```python
def decimal_to_base(num, base):
    if num == 0:
        return "0"
    # Digits mapping up to Base 36
    chars = "0123456789abcdefghijklmnopqrstuvwxyz"
    result = ""
    while num > 0:
        result = chars[num % base] + result
        num //= base
    return result

# Example: Convert 22 to Base 5
print(decimal_to_base(22, 5))   # Output: "42"

# Example: Convert 1295 to Base 36
print(decimal_to_base(1295, 36)) # Output: "zz"
```

Use code with caution.

Any Base → Base 10 (Decimal)

Use `Integer.parseInt(string, radix)`.

java

```java
// Convert Base 7 string "15" to Decimal (1*7^1 + 5*7^0 = 12)
int base7ToDecimal = Integer.parseInt("15", 7);
System.out.println(base7ToDecimal); // Output: 12

// Convert Base 36 string "abc" to Decimal
int base36ToDecimal = Integer.parseInt("abc", 36);
System.out.println(base36ToDecimal); // Output: 13368
```

Use code with caution.

Base 10 (Decimal) → Any Base

Use `Integer.toString(int, radix)`.

java

```java
int decimalNum = 13368;

// Convert 13368 to a Base 36 string
String base36Str = Integer.toString(decimalNum, 36);
System.out.println(base36Str); // Output: "abc"

// Convert 12 to a Base 7 string
String base7Str = Integer.toString(12, 7);
System.out.println(base7Str);   // Output: "15"
```

Use code with caution.

Time Complexity

input size	required time complexity
$n \leq 10$	$O(n!)$
$n \leq 20$	$O(2^n)$
$n \leq 500$	$O(n^3)$
$n \leq 5000$	$O(n^2)$
$n \leq 10^6$	$O(n \log n)$ or $O(n)$
n is large	$O(1)$ or $O(\log n)$